Zero Trust Architecture with Bias Risk and Grounding Controls for AI Interview Systems

Sai Pranavi A P, Sai Triveni B, Sanjana H V, and Sneha V
Department of AI & ML, RV College of Engineering
Bengaluru, India

***Abstract*—AI interview systems must govern access to candidate records and the release of model-generated feedback. Authorization alone does not establish that an assessment is supported or free from discriminatory treatment. This paper presents TrustInterview AI, an architecture that separates access assurance from output assurance and couples their decisions at assessment transitions. The access plane checks provisioned identity, role, tenant, ownership and session validity. The output plane checks source provenance, selected sensitive-content signals, claim support and control availability before release, review or abstention. A controlled reference implementation provides server-owned assessments, immutable resume evidence, transactional persistence and authorized review. Sixty-nine backend tests and twelve analysis-tool tests pass; a forty-eight-case offline pilot verifies experiment recording without estimating efficacy. The implemented claim checker handles a bounded technology-use grammar, while an optional model adapter remains unverified with a live provider. We specify independent annotation and paired demographic experiments to evaluate the remaining coverage and fairness questions. These results establish selected enforcement behaviours, not general hallucination reduction or demographic fairness.**

***Index Terms*—Zero Trust Architecture, AI interviews, bias risk, source grounding, output assurance, policy enforcement**

I. Introduction

An AI interview workflow handles several kinds of authority. A candidate may submit an answer, an application may retrieve a resume, a model may propose feedback, and a reviewer may approve its release. These actions cross different trust boundaries. A valid session can request another candidate's record, while an authorized model request can return an unsupported assessment. A detector warning can also be ineffective when the application persists or displays the warned output. The resulting research problem concerns the conditions under which an artifact may move through the workflow, rather than the presence of security middleware alone.

Zero Trust Architecture (ZTA) provides a resource-centred basis for access decisions. NIST SP 800-207 requires the system to avoid treating network location as sufficient trust and to evaluate access to protected resources [1]. Applying this principle to an interview requires server-side subject–resource authorization, including the candidate's ownership of an assessment. However, ZTA does not by itself evaluate factual support or hiring bias. We therefore introduce an application-level output-assurance plane alongside the access plane. A model response becomes input to a release decision, not authoritative evidence of its own correctness.

The output problem has two distinct forms. A model can invent a qualification or recommendation that the supplied evidence does not support. Separately, a candidate's statement can differ from the resume. The latter is a source discrepancy, not automatically a model hallucination or proof of dishonesty. A resume is candidate-supplied material and does not independently verify qualifications. Bias introduces another distinction: mentioning a sensitive attribute is different from treating otherwise comparable candidates differently. Both source grounding and sensitive-content screening therefore require explicit evidence boundaries.

A. Objective and Contributions

We investigate how resource authorization and output-release controls can be coupled in an AI interview without collapsing them into one risk score. The proposed contribution is an enforceable architecture: each plane produces an explicit decision, and assessment state records whether the result is released, held, rejected or withheld. Our work has three parts:

1) A threat model and dual-policy architecture connecting identity, resource ownership, bias-risk evidence and grounding at interview-workflow transitions.
2) A controlled prototype implementing selected access and output invariants, including immutable source binding, transactional assessment state and reasoned human review.
3) A reproducible verification artifact and a prepared annotation and counterfactual protocol that distinguish conformance tests from efficacy measurements.

The paper is a systems and architecture study with preliminary implementation verification. We do not claim a new general-purpose fairness or hallucination detector. The current score is a narrow keyword fixture, not a validated measure of interview competence. An optional live-provider adapter has been implemented but tested only with transport fixtures. The remaining empirical questions concern semantic coverage, group differences in release burden and actual provider behaviour. They are stated as evaluation requirements rather than reported findings.

II. Related Work

A. Zero Trust and Generative AI Risk

NIST SP 800-207 identifies a policy decision point (PDP) and a policy enforcement point (PEP) for resource access [1].

SP 800-207A extends the access-control discussion to cloud-native and multi-cloud applications [2]. These sources motivate checking subject, resource and context at an application boundary. They do not define a hiring-fairness test or establish the semantic correctness of a model assessment. NIST's Generative AI Profile separately identifies risks requiring evaluation and monitoring, including confabulation and harmful bias [3]. Our architecture connects these concerns operationally while retaining their different decision criteria.

B. Bias in Hiring Workflows

Research on language-model hiring decisions evaluates demographic effects using controlled candidate comparisons [4]. Name-based employment recommendations also illustrate why demographic signals can affect outputs even when qualifications are comparable [5]. JobFair provides a framework for benchmarking gender hiring bias [6]; other job–resume matching work examines gender, race and education [7]. Resume language itself can carry demographic signals, complicating a simple removal strategy [8].

These studies motivate paired experiments and subgroup error analysis. A lexical screen can flag a quotation containing a selected sensitive term, but cannot determine whether a hiring decision is discriminatory. It can miss indirect language and can burden candidates who disclose an attribute without making it job-relevant. We use such findings to trigger review and reserve the term fairness for a separate comparison of outcomes under controlled variations.

C. Grounding and Adversarial Evidence

Hallucination research distinguishes generated claims, evidence and evaluation methods [9]. SelfCheckGPT uses consistency across samples to identify potentially unreliable text [10]; consistency alone does not establish that a claim about a candidate follows from a resume. Our source-relative formulation instead asks whether a particular statement is supported, contradicted or undetermined by the supplied evidence, without treating that evidence as independently true.

Candidate material is also an adversarial input boundary. Resume prompt-injection studies examine attempts to redirect automated screening [11], [12], while OWASP identifies prompt injection among application risks [13]. Accordingly, candidate text cannot change role, ownership, rubric or release policy. The provider receives evidence as data, and the server validates returned artifacts. Prompt instructions complement this enforcement but do not replace it.

D. Position of This Work

Our contribution is the coupling between resource authorization and evidence-based artifact release. Access-control literature, hiring-bias benchmarks and grounding methods inform different parts of this design. The prototype makes their interaction visible through server-owned state and auditable outcomes. Its novelty must be assessed at this architectural boundary, rather than inferred from a count of middleware layers or configured detector categories. General semantic grounding and hiring fairness remain empirical tasks requiring independent labels and real model outputs.

TABLE I
THREATS AND REQUIRED ENFORCEMENT OUTCOMES

Threat	Required outcome
Foreign resource access	Deny candidate or reviewer access outside the authorized ownership/tenant scope.
Client identity or score substitution	Ignore client authority fields; derive principal and score on the server.
Replay or concurrent mutation	Permit only valid versioned assessment transitions.
Unsupported provider artifact	Abstain before scored output is released.
Sensitive or instruction-like output	Hold for authorized review; do not treat a warning as prevention.
Provider or store failure	Withhold results; preserve failure and recovery evidence.

III. Terminology and Threat Model

A. Actors, Assets and Assumptions

The protected assets are resume text, answers, assessment state, scores, proposed feedback and review records. Candidates access their own assessments within an assigned tenant; reviewers inspect held assessments within their tenant. The browser, candidate material and model responses are untrusted inputs. The backend policy implementation, provisioned principal registry and authoritative store form the trusted computing base. Compromise of these components is outside the prototype's demonstrated protection.

Our reference deployment is a single backend instance with exclusive ownership of its SQLite database. Sessions are volatile and require new authentication after restart. Identity can be provisioned locally or mapped from a verified external subject, but real Google sign-in has not been exercised. Tenant and role bindings remain server-owned. No biometric identity, proctoring or identity inference from names is included.

B. Output Definitions

Source-supported means the available evidence supports a statement in the declared scope. Source-contradicted requires explicit incompatible evidence in the same scope. Undetermined covers missing, insufficient, ambiguous or conflicting evidence. Absence of a technology in a resume is not contradiction, and a supported resume quotation is not external qualification verification. Model-origin unsupported assertions and candidate-origin source inconsistencies are recorded separately.

Bias-risk evidence is a signal for inspection, not a discrimination finding. In the prototype, selected sensitive-attribute terms and instruction-like phrases are deterministic review triggers. Their vocabulary and contextual limits are explicit. Operational abstention is also separate from the semantic label undetermined: a provider failure can cause abstention without producing a claim label.

Table I identifies workflow threats rather than an exhaustive attack taxonomy. An attacker may reuse a stolen bearer token

until expiry or revocation; the prototype does not demonstrate device binding or resistance to a compromised browser. Likewise, database hashes provide local consistency checks, not protection against an administrator rewriting the store and its audit history. These limits define which threat claims the tests can support.

IV. Dual Policy Zero Trust Architecture

A. Decision Separation

Figure 1 shows the two assurance planes. The access plane decides whether a principal may act on a resource. The output plane decides whether the resulting artifact may be released. The latter is an application policy extension, not a claim that NIST defines fairness as an access-control property. We express the decisions as

$$D_A = f_A(I, R, T, O, S, X), \tag{1}$$

$$D_O = f_O(A, E, B, G, P, H), \tag{2}$$

where I is identity, R role, T tenant, O ownership, S session status and X assessment context. For output, A is the proposed artifact, E its bound evidence, B bias-risk signals, G grounding results, P provenance and H control availability. A release transition requires valid access and an acceptable output decision. Neither a successful provider call nor a detector score grants resource authority.

The architectural access outcomes include allow, deny, step-up and revoke. The current reference service implements allow/deny with expiry and revocation; interactive step-up is not implemented. The output design admits release, regeneration, abstention, review and denial. The implemented path releases valid output, holds selected findings for review, or abstains; regeneration is not implemented. Stating these differences avoids presenting every conceptual policy outcome as an existing feature.

B. Resource and Session Enforcement

The backend resolves a principal from a provisioned credential or trusted identity adapter. It issues an opaque session token and keeps expiry and revocation state on the server. Protected routes require that token. For candidates, every assessment operation checks role, tenant and owner; reviewers may inspect only held resources in their tenant. Inaccessible resources return a non-disclosing resource error. A role string or user identifier submitted in JSON cannot replace these bindings.

Assessments are created with a server-owned question and rubric. Submission accepts an answer rather than a client score, and begins a versioned transition from awaiting-answer to evaluating. Concurrent or repeated submissions cannot overwrite committed state. After asynchronous provider work, session validity is checked again before release. A revoked or expired session cannot gain a scored result simply because the provider completed successfully.

The external identity adapter uses browser-bound state, nonce, Proof Key for Code Exchange (PKCE) and a server-assigned subject mapping following Google OpenID Connect documentation [14]. Signed local test assertions exercise cryptographic validation and negative claims; code exchange and key retrieval are fixtures. This establishes selected protocol behaviour, not successful live authentication. Reviewer access still derives from the server's binding rather than provider-supplied role text.

C. Source Binding and Grounding

An optional plain-text resume is attached once before submission. The input is bounded to 8000 UTF-8 bytes, rejects unexpected authority fields and invalid text, and belongs to the candidate's assessment. The store records its text and SHA-256 digest with a correlated audit event. The digest is checked before provider invocation and enters the cache context. Evidence is copied for the provider, preventing its in-process mutation from becoming authoritative source state. PDF ingestion and OCR are not implemented in this controlled path.

The feedback contract permits bounded exact quotations linked to answer, question or resume. Unknown source identifiers, fabricated quotations and unexpected output fields cause abstention. This is structural provenance checking: a quoted falsehood can pass, a relevant phrase can omit context, and correct paraphrases cannot pass a quotation-only contract. Accordingly, quote matching is never reported as general semantic hallucination detection.

The current claim checker adds a deliberately limited source comparison. It recognizes whole-line first-person technology-use statements and their explicit negations, using a fixed alias vocabulary. Same-polarity evidence is source-supported; opposite-polarity evidence is source-contradicted. Missing assertions, conflicting statements, unknown technologies and unsupported wording remain undetermined. Java and JavaScript are distinct, while aliases such as JS and JavaScript are explicitly mapped. General prose, dates, duration and competence require a broader semantic method. When no claim is extracted, the input is outside the checker's recognized scope; it is not verified.

D. Bias-Risk Routing and Release

The output screen checks quoted text for selected sensitive-attribute and instruction-like phrases using word boundaries. A finding routes the output to review rather than asserting discrimination. This prevents certain flagged artifacts from being silently released but leaves indirect language, context and vocabulary coverage unresolved. Source provenance is retained so that a candidate disclosure is not automatically treated as a model's demographic judgment.

Table II defines precedence: review cannot override an abstained grounding failure. A same-tenant reviewer can approve or reject a held result only by recording a reason. Approval authorizes release of that artifact; it does not verify a qualification or validate the scoring rubric. Rejection withholds the score. Candidate displays contain scores and feedback only after durable release, including when a malicious response contains a score field in a held state.

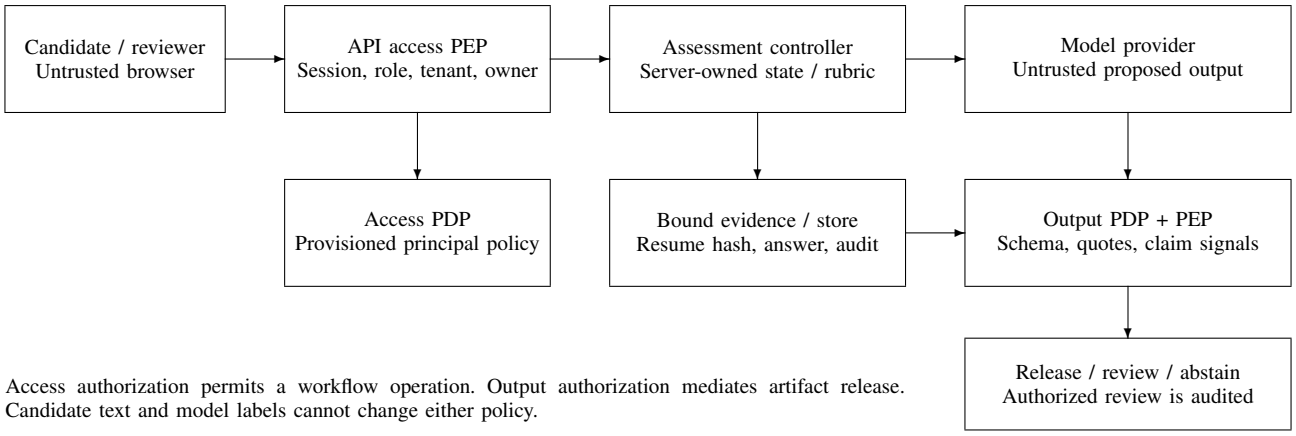


Fig. 1. Coupled assurance planes. Solid arrows show processing or evidence dependencies; all protected reads and mutations remain mediated by the access PEP.

TABLE II
IMPLEMENTED OUTPUT-POLICY PRECEDENCE

Condition	Action
Malformed output, unavailable source or fabricated quote	Abstain
Extracted claim has undetermined support	Abstain
Source contradiction or selected risk trigger, with otherwise valid output	Hold for review
No blocking condition in the implemented scope	Release
Provider, session or durable-commit failure	Withhold result

E. Persistence, Audit and Failure Handling

Assessment mutation and its audit event commit in one SQLite transaction. Compare-and-swap version checks prevent stale updates. A crash during provider evaluation leaves durable evaluating state; startup recovery changes interrupted assessments to abstained and removes uncommitted result fields. Exclusive database ownership prevents a second instance from recovering another active instance's work. The design is a single-instance reference implementation, not a distributed availability claim.

Audit records contain policy version, correlated request, action and decision, hashed principal references and selected source/review hashes. A hash chain detects some local alterations but lacks external anchoring and cannot prevent privileged rewriting or valid-prefix truncation. Resume and answer evidence remain plaintext in the database. Sessions and caches are volatile. Cache writes follow durable release and use candidate/session/assessment/source context; cache reuse and eviction have not been evaluated.

V. PROTOTYPE IMPLEMENTATION AND BOUNDARIES

A. Historical Prototype and Controlled Reference

The historical INTERVIEW-BOT and TrustInterview-AI repositories supply implementation context, not the final research evidence. A code review identified anonymous session issuance, inconsistent ownership checks, client-authoritative scores, shared question context, route-dependent detector behaviour and simulated interface values. Missing resume evidence could also be interpreted as verified by a heuristic path. These observations motivated an isolated reference boundary rather than a claim that the original middleware already established ZTA.

The controlled backend is the default entry point and serves a dedicated candidate/reviewer interface. Historical routes remain available only through an explicit legacy entry point and are not mounted by the controlled service. Table III summarizes what has been implemented and what remains limited. Backend and analysis sources, test reports and versioned input files are retained with SHA-256 manifests. The long traceability review and test logs are supporting artifacts; their detail is not counted as a separate research contribution for each remediation step.

The fixture score is the rounded percentage of the terms *identity*, *authorization* and *resource* occurring as words in the answer. Term presence, rather than repetition or resume content, determines this value. This transparent rule permits reproducible authority tests but supplies no evidence of competence.

B. Model Adapter and Recording Protocol

The optional Gemini REST adapter uses a fixed service endpoint and privately configured credentials. It transmits bounded evidence, requests a constrained JSON schema and invokes no tools. Timeouts abort requests; HTTP errors, blocked or incomplete responses, malformed JSON and oversized bodies fail closed. Live mode has no automatic offline fallback. The default provider remains an explicitly labelled offline fixture. The format follows the provider documentation [15]; fixture tests do not establish live API interoperability or reliability.

A sequential experiment runner uses a bounded seeded plan and a fresh output directory. It records input and prompt-template hashes, provider/model and decoding metadata, re-

TABLE III
REFERENCE IMPLEMENTATION AND EVIDENCE BOUNDARIES

Component	Implemented boundary / remaining limitation
Identity and access	Provisioned subjects, scoped resources, expiry and revocation; live external sign-in untested.
Resume evidence	Owned immutable text and digest; no PDF/OCR or qualification verification.
Output assurance	Exact quotations, bounded claim grammar and review triggers; no validated general detector.
Persistence and review	Transactional state, recovery and reasoned reviewer decisions; single-instance plaintext store.
Scoring	Server-owned three-term keyword fixture; no competence or hiring validity.
Model integration	Optional bounded Gemini adapter; transport fixtures only, no live interoperability result.

TABLE IV
CURRENT VERIFICATION ARTIFACT

Check	Recorded outcome
Backend controls, identity, persistence, grounding, provider and runner	69 tests passed
Candidate/reviewer and mode-disclosure interface	5 tests passed; build succeeded
Annotation schema and metric arithmetic	12 artificial tests passed
Synthetic offline recording pilot	48 complete records; 0 live calls
Independent annotations and live efficacy	Not collected

quest identifiers, response references, token usage when supplied, duration and decisions. Successful bounded responses retain their envelopes; failed or rejected requests retain sanitized diagnostics. Partial runs remain marked incomplete. This filesystem recorder is not the transactional assessment store and does not promise power-loss durability.

Grounding predictions and paired-output records are exported separately. A live semantic label would remain an untrusted prediction even when its cited quotation exists. The paired workflow keeps the fixed ZTA question and keyword score; job text does not alter that rubric. Its appropriate outcomes are feedback-source selection and release, abstention or review burden. It cannot measure model-driven candidate ranking or general hiring-score fairness.

VI. VERIFICATION AND EVALUATION PROTOCOL

A. Current Verification Results

The retained verification environment uses Node.js v22.20.0 on Windows x64 for the controlled backend and a separate Python analysis utility. Table IV reports the current checks. Test inputs were authored to exercise selected policy and schema conditions; they were not sampled from a threat population or independently labelled applicant corpus. Passing counts therefore establish conformance for those inputs, not accuracy estimates.

Access tests exercise unprovisioned identities, isolation by role, tenant and owner, expiry, revocation, replay, concurrent submission and client-score rejection. Output tests exercise unsupported quotations, extra model fields, review authorization and withheld scores. Persistence tests check atomic commits of state and audit events, stale versions, exclusive ownership and abrupt process exit during a transaction. Reopening the crash fixture rolls back a fake uncommitted release and records abstention for interrupted evaluation. Identity tests verify locally signed RSA assertions with the installed SDK and reject altered signatures and invalid issuer, audience, nonce and expiry claims. They do not use a live Google account.

Provider tests use injected transport fixtures to check request boundaries, secret handling, timeout abort and failure outcomes. The forty-eight-case runner smoke check retains twenty-four grounding and twenty-four paired records. Raw hashes, complete case coverage and cross-language input hashes validate against the annotation importer. These records are explicitly offline, and no prediction-to-gold analysis is performed because human labels remain blank. The keyword score ignores demographic text and source discrepancies; equal scores under this fixture are expected by construction and are not a fairness result.

TABLE V
HISTORICAL COMPONENT TIMING (MILLISECONDS)

Configuration	p50	p95	p99
C0	0.0013	0.0027	0.0053
C1	0.0052	0.0116	0.0283
C2	0.0029	0.0065	0.0240
C3	0.0160	0.0386	0.1458

B. Historical Component Timing

An earlier frozen reference snapshot contains seventeen tests, fourteen output-contract fixtures and a synchronous component benchmark. That source snapshot is retained separately; its timing must not be attributed to the later SQLite, identity or provider integration. On an Intel Core i5-1155G7, each configuration ran 1000 measured iterations after 50 warm-up rounds, in seeded randomized order. C0 applied the keyword scorer, C1 added session/ownership checks, C2 added the output contract and C3 combined checks with in-memory audit recording. Table V reproduces those historical values.

The benchmark excludes HTTP, database writes, model inference, reviewer work and concurrent traffic. It supports only component-level observations for the frozen fixture. No end-to-end overhead or deployment latency is inferred from it. A future integrated measurement must record access, evidence extraction, provider and commit time separately and retain failed calls rather than timing only successful completions.

C. Research Questions and Study Configurations

The efficacy study asks four questions. RQ1 asks whether resource and transition controls prevent unauthorized reads and scored releases under adversarial requests. RQ2 asks how accurately the grounding method distinguishes supported, contradicted and undetermined claims, including extraction failures.

RQ3 asks whether output decisions and error burdens change across qualification-preserving demographic pairs. RQ4 asks what latency, failure and review cost the coupled controls introduce under a specified workload.

The proposed configurations isolate contributions: C0 is a declared baseline, C1 adds access assurance, C2 adds output assurance, C3 combines both, and C4 adds the selected advanced semantic method. They must share the same provider, rubric, input set and decoding settings when the comparison requires them. The historical timing configurations are component analogues, not evidence that these live system ablations have been run. Any control disabled for a synthetic experiment remains separate from a deployed protected workflow.

D. Prepared Corpora and Independent Annotation

The synthetic procedure pilot has twenty-four grounding examples and twelve two-member demographic pairs. Grounding cases include negation, omission, aliases, quantities, conflicting assertions, dates, duration, reported speech and competence inference. Claim origin distinguishes candidate source discrepancy from unsupported model assertions. The cases are authored inputs, not observed provider outputs. Annotators judge general semantic support beyond the implemented grammar, allowing coverage gaps to appear in the evaluation.

Pairs cover three junior technical roles and four declared signals: pronouns, age, marital status and disclosed disability. Within each pair, the job, technical evidence and answer are identical. These signals require faculty review before a larger study and do not represent a population's distribution or legitimate hiring criteria. Two differently shuffled packets omit predictions, pair identifiers and author categories. Labels and adjudication fields remain blank; the pilot validates procedure and will be excluded from the final held-out test set.

Two human reviewers independently label claims as source-supported, source-contradicted or undetermined, citing exact evidence spans and recording ambiguity. Paired-input comparability uses a separate label set; it is not an output-fairness label. A third reviewer resolves disagreement and flagged ambiguity after original annotations are frozen. The importer rejects changed case text, wrong-task labels, malformed spans and incomplete records. Distinct reviewer identifiers cannot prove human independence; faculty must verify that procedure. Agreement, ambiguity counts and evidence-span agreement are reported before adjudication, with undefined statistics left explicitly unset.

E. Metrics, Analysis and Reproducibility

Security evaluation counts unauthorized transitions, replay acceptance, cross-tenant leakage, tampered-score acceptance and audited decisions. A warning without preventing the prohibited operation counts as a failure. Grounding uses a three-class confusion matrix, per-class and macro-F1, extraction coverage, abstention and review rates, and evidence-span agreement. Candidate-origin and model-origin claims are reported separately. Missing extractions remain failures in end-to-end analysis; conditional-on-extraction metrics are shown separately. An empty extracted subset receives no successful accuracy value.

Fairness analysis compares paired output scores only when the rubric is meaningful and the scores are available. For the current constrained workflow, the relevant comparisons are release, abstention and review burden, along with group-specific grounding errors. Missing or held scores are not replaced by zero. Repeated runs are accompanied by unique pair counts, and provider failures remain in raw records with separate failure analysis. Ranking or selection-rate claims require a defined ranking task and a predeclared selection threshold; neither is evaluated here.

For a final powered study, sample size, primary outcomes and analysis rules must be fixed before collection. The protocol calls for uncertainty intervals, paired analysis and multiplicity correction where appropriate; the present descriptive utility does not implement population inference. Pilot cases used to estimate variance or refine instructions must not enter the final test set. A zero observed failure count must be accompanied by its sample size and uncertainty, not a claim of zero risk.

Each report records source/input hashes, case identifiers, model/configuration metadata, raw response references, decisions and error states. Raw artifacts must be reviewed without exposing credentials or applicant data. Tables for efficacy will be generated from frozen annotations and prediction exports after collection. No such tables are populated in this paper. At present, the completed evidence consists of implementation verification and an offline recording protocol, with no live model calls or independent efficacy labels.

VII. Discussion and Limitations

The strongest current result is an inspectable release boundary. A browser cannot authoritatively select a tenant, reviewer role or assessment score. A model cannot add a score field, cite an absent source or bypass a failed durable commit while remaining within the demonstrated workflow. Selected sensitive signals and source contradictions create an auditable review transition. This makes the interaction between access authority and output authority concrete, which is the principal architectural contribution.

The checks leave substantial semantic limitations. Quotation matching verifies a substring relationship, not entailment, external truth, salience or completeness. The controlled grammar misses general resume prose and does not model changes over time. A quoted sensitive disclosure may cause unnecessary review, whereas indirect discriminatory language can escape the trigger vocabulary. A reviewer can authorize an artifact without establishing its fairness. Consequently, neither test counts nor the number of rules supports a claim that bias or hallucination has been eliminated.

Operational generalization is also limited. The reference backend is single-instance, sessions are memory-resident, evidence is plaintext, and the audit chain is not externally anchored. The optional provider has not been exercised live, and timeout or model-version effects have not been measured in deployment. The synthetic pilot contains no real applicants

and only selected language, roles and demographic signals. Any later use of candidate data requires institutional approval, appropriate participation and data minimization, with a process to contest outcomes. The current score must not be used for autonomous employment decisions.

For faculty review, the architecture, code boundary and preliminary verification form a complete description of the present artifact. A stronger empirical submission requires independent annotation, a meaningful task rubric and recorded live provider outputs. These are missing research evidence, not formatting details. Further claims should follow that evidence rather than be inferred from a working interface or a successful API integration.

VIII. Conclusion

The TrustInterview AI architecture couples resource access and feedback release through server-owned assessment state. It binds candidate evidence, validates constrained provider artifacts, supports reasoned review and preserves selected failure transitions transactionally. Verification records 69 passing backend tests, five interface tests and twelve analysis-tool tests. The 48 offline pilot records establish recording and importer compatibility. These findings concern selected workflow behaviours; general semantic grounding, demographic fairness, competence scoring and live-provider reliability remain unvalidated. Independent annotation and paired model experiments are required to test whether the architecture improves AI interview outcomes beyond enforcing its explicit boundaries.

Acknowledgment

We thank Prof. Apoorva Chate for guidance on this project.

References

[1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020.

[2] R. Chandramouli and Z. Butcher, "A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Cloud Environments," NIST Special Publication 800-207A, Sep. 2023, doi: 10.6028/NIST.SP.800-207A.

[3] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, Jul. 2024, doi: 10.6028/NIST.AI.600-1.

[4] H. An, C. Acquaye, C. Wang, Z. Li, and R. Rudinger, "Do Large Language Models Discriminate in Hiring Decisions on the Basis of Race, Ethnicity, and Gender?" in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics, Short Papers*, 2024, pp. 386–397, doi: 10.18653/v1/2024.acl-short.37.

[5] H. Nghiem, J. Prindle, J. Zhao, and H. Daumé III, "You Gotta be a Doctor, Lin: An Investigation of Name-Based Bias of Large Language Models in Employment Recommendations," in *Proc. 2024 Conf. Empirical Methods in Natural Language Processing*, 2024, pp. 7268–7287, doi: 10.18653/v1/2024.emnlp-main.413.

[6] Z. Wang *et al.*, "JobFair: A Framework for Benchmarking Gender Hiring Bias in Large Language Models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 3227–3246, doi: 10.18653/v1/2024.findings-emnlp.184.

[7] H. Iso, P. Pezeshkpour, N. Bhutani, and E. Hruschka, "Evaluating Bias in LLMs for Job-Resume Matching: Gender, Race, and Education," in *Proc. 2025 Conf. of the Nations of the Americas Chapter of the Association for Computational Linguistics, Industry Track*, 2025, pp. 672–683, doi: 10.18653/v1/2025.naacl-industry.55.

[8] P. Parasurama and J. Sedoc, "Gendered Language in Resumes and its Implications for Algorithmic Bias in Hiring," in *Proc. 4th Workshop on Gender Bias in Natural Language Processing*, 2022, p. 74, doi: 10.18653/v1/2022.gebnlp-1.7.

[9] Y. Zhang *et al.*, "Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models," *Computational Linguistics*, vol. 51, no. 4, pp. 1373–1418, 2025, doi: 10.1162/coli.a.16.

[10] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing*, 2023, pp. 9004–9017, doi: 10.18653/v1/2023.emnlp-main.557.

[11] P. Baxi, J. Xu, J. Y. Jiang, and S. Jasin, "Prompt Injection in Automated Résumé Screening with Large Language Models: Single and Multi-Injection Settings," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 2942–2953, doi: 10.18653/v1/2026.findings-acl.142.

[12] Y. Augey, J. H. Levy, and A. Akdemir, "RAPIDS: Resume Attack Prompt Injection Detection at Scale," in *Proc. 64th Annual Meeting of the Association for Computational Linguistics, Industry Track*, 2026, pp. 1848–1865, doi: 10.18653/v1/2026.acl-industry.127.

[13] OWASP Foundation, "OWASP Top 10 for LLM Applications 2025," OWASP GenAI Security Project, 2025. [Online]. Available: https://genai.owasp.org/llm-top-10/

[14] Google, "OpenID Connect," Google Identity documentation. [Online]. Available: https://developers.google.com/identity/openid-connect/openid-connect. Accessed: Oct. 11, 2026.

[15] Google, "Structured outputs," Google AI for Developers documentation. [Online]. Available: https://ai.google.dev/gemini-api/docs/generate-content/structured-output. Accessed: Oct. 11, 2026.